COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	September 10, 2026	**DATE SUBMITTED:**	September 10, 2026
NAME:	Gresola, Aryanna Keith G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.1 **Sorting Algorithm Part 2**

1. PROCEDURE OUTPUT

INPUT - PREPARATION TASK

‘

sorting.h

```c
#ifndef SORTING_H
#define SORTING_H
#include <iostream>

// display functions of random generated values:
template <typename T>
void display(T arr[], int size){
    for(int i = 0; i < size; i++){
        std::cout << arr[i] <<" ";
        if((i + 1) % 10 == 0){
            std::cout << std::endl;
        }
    }
}
#endif
```

main.cpp

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

int main(){
    const int SIZE = 100;
    int numbers[SIZE];
    srand(time(0));

    // generating 100 random numbers:
    for(int i  = 0; i < SIZE; i++){
        numbers[i] = rand() % 100 + 1;
    }

    std::cout << "--- ORIGINAL UNSORTED ARRAY ---" << std::endl;
    display(numbers, SIZE);

    return 0;
}
```

OUTPUT

```
--- ORIGINAL UNSORTED ARRAY ---
91 23 47 98 14 83 67 12 50 29
36 36 92 40 26 70 83 44 3 94
20 40 9 21 99 54 12 89 77 89
55 12 78 29 90 86 62 4 8 74
66 67 30 13 68 28 47 85 39 15
65 10 99 59 1 18 85 62 34 62
65 88 35 19 29 92 73 78 53 15
87 52 80 73 53 85 54 20 20 35
2 92 88 85 18 64 41 44 56 79
58 54 12 27 86 9 71 24 57 58
```

OBSERVATION/ANALYSIS

The program first generates 100 random numbers and stores them in the array. Since SIZE is set to 100, the loop in main.cpp runs 100 times. After generating the numbers, the display() function also goes through all 100 elements to show them on the screen. At this part, there are still no sorting passes or comparisons

because the program is only generating and displaying the numbers. The loops simply run n times, so when n = 100, there are 100 iterations. The time complexity for both generating and displaying the values is O(n) because each element is processed once. Therefore, for 100 elements, both loops perform 100 iterations. This part only prepares the data that will be used by the sorting algorithms, so the actual sorting passes and comparisons happen after this step.

INPUT - SHELL SORT

sortingAlgo.h

```c
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

// shell sort:
template <typename T>
void shellSort(T arr[], int size){
    for(int interval = size / 2; interval >= 1; interval = interval / 2){
        for(int i = interval; i < size; i++){
            T temp = arr[i];
            int j = i;

            while(j >= interval && arr[j - interval] > temp){
                arr[j] = arr[j - interval];
                j = j - interval;
            }
            arr[j] = temp;
        }
    }
}

#endif
```

main.cpp

```c
    std::cout << "--- SORTED ARRAY ---" << std::endl;
    std::cout << "Shell Sort: " << std::endl;
    shellSort(numbers, SIZE);
    display(numbers, SIZE);
    std::cout << std::endl;

    return 0;
}
```

OUTPUT

```
--- SORTED ARRAY ---
Shell Sort:
1 2 3 4 8 9 9 10 12 12
12 12 13 14 15 15 18 18 19 20
20 20 21 23 24 26 27 28 29 29
29 30 34 35 35 36 36 39 40 40
41 44 44 47 47 50 52 53 53 54
54 54 55 56 57 58 58 59 62 62
62 64 65 65 66 67 67 68 70 71
73 73 74 77 78 78 79 80 83 83
85 85 85 85 86 86 87 88 88 89
89 90 91 92 92 92 94 98 99 99
```

OBSERVATION/ANALYSIS

The Shell Sort starts after the 100 random numbers have been generated and displayed. The shellSort() function uses different intervals to compare and arrange the elements. Since SIZE is set to 100, the first interval is 50, then it is divided by 2 until it reaches 1. The intervals are 50, 25, 12, 6, 3, and 1, so there are 6 passes before the list becomes fully sorted. In each pass, the inner for loop goes through the elements, while the while loop compares the values and shifts them whenever needed. The exact number of comparisons and shifts can be different because the numbers are randomly generated. The time complexity of Shell Sort depends on the interval sequence used. For this code, the best case is O(n log n), while the worst case is O(n^2). After the 6 passes, the array is sorted, and the display() function goes through all 100 elements again to show the correct sorted list.

INPUT - MERGE SORT

sortingAlgo.h

```
22  template <typename T>
23  void merge(T arr[], int left, int middle, int right){
24      T temp[100];
25      int i = left;
26      int j = middle + 1;
27      int k = 0;
28
29      while(i <= middle && j <= right){
30          if(arr[i] <= arr[j]){
31              temp[k] = arr[i];
32              i++;
33          }else{
34              temp[k] = arr[j];
35              j++;
36          }
37          k++;
38      }while(i <= middle){
39          temp[k] = arr[i];
40          i++;
41          k++;
42      }while(j <= right){
43          temp[k] = arr[j];
44          j++;
45          k++;
46      }
47      for(i = left, k = 0; i <= right; i++, k++){
48          arr[i] = temp[k];
49      }
50  }
51
52  template <typename T> // this the function that split array in half
53  void mergeSort(T arr[], int left, int right){
54      if(left < right){
55          int middle = (left + right) / 2;
56
57          mergeSort(arr, left, middle);
58          mergeSort(arr, middle + 1, right);
59          merge(arr, left, middle, right);
60      }
61  }
```

main.cpp

```
28      // merge sort:
29      std::cout << "Merge Sort: " << std::endl;
30      mergeSort(numbers, 0, SIZE - 1);
31      display(numbers, SIZE);
32      std::cout << std::endl;
33
34      return 0;
35  }
```

OUTPUT

```
Merge Sort:
1 2 3 4 8 9 9 10 12 12
12 12 13 14 15 15 18 18 19 20
20 20 21 23 24 26 27 28 29 29
29 30 34 35 35 36 36 39 40 40
41 44 44 47 47 50 52 53 53 54
54 54 55 56 57 58 58 59 62 62
62 64 65 65 66 67 67 68 70 71
73 73 74 77 78 78 79 80 83 83
85 85 85 85 86 86 87 88 88 89
89 90 91 92 92 92 94 98 99 99
```

OBSERVATION/ANALYSIS

The Merge Sort starts after the 100 random numbers have been generated and displayed. The mergeSort() function first splits the 100 elements into two parts, then keeps splitting them into smaller parts until each part has only one element. After that, the merge() function starts comparing and combining the elements back together in the correct order. For 100 elements, it takes about 7 levels of splitting before reaching the individual elements, since the array is repeatedly divided by 2. The exact number of comparisons can vary depending on the random values generated. The time complexity of Merge Sort is O(n log n) for both the best and worst case, because it still needs to split and merge the elements even if the numbers are already sorted. After all the parts are merged, the array is completely sorted, and display() goes through all 100 elements again to show the final result.

INPUT - QUICK SORT

sortingAlgo.h

```
63  // quick sort:
64  template <typename T> // function that splits array
65  int arrPartition(T arr[], int low, int high){
66      T pivot = arr[high];
67      int i = low - 1;
68
69      for(int j = low; j < high; j++){
70          if(arr[j] < pivot){
71              i++;
72
73              T temp = arr[i];
74              arr[i] = arr[j];
75              arr[j] = temp;
76          }
77      }
78      T temp = arr[i + 1];
79      arr[i + 1] = arr[high];
80      arr[high] = temp;
81
82      return i + 1;
83  }
```

```
84
85  template <typename T>
86  void quickSort(T arr[], int low, int high){
87      if(low < high){
88          int pivot = arrPartition(arr, low, high);
89
90          quickSort(arr, low, pivot - 1);
91          quickSort(arr, pivot + 1, high);
92      }
93  }
```

main.cpp

```
34      // quick sort:
35      std::cout << "Quick Sort: " << std::endl;
36      quickSort(numbers, 0, SIZE - 1);
37      display(numbers, SIZE);
38      std::cout << std::endl;
```

OUTPUT

```
Quick Sort:
1 2 3 4 8 9 9 10 12 12
12 12 13 14 15 15 18 18 19 20
20 20 21 23 24 26 27 28 29 29
29 30 34 35 35 36 36 39 40 40
41 44 44 47 47 50 52 53 53 54
54 54 55 56 57 58 58 59 62 62
62 64 65 65 66 67 67 68 70 71
73 73 74 77 78 78 79 80 83 83
85 85 85 85 86 86 87 88 88 89
89 90 91 92 92 92 94 98 99 99
```

OBSERVATION/ANALYSIS

The Quick Sort starts after the 100 random numbers have been generated and displayed. The quickSort() function uses the last element as the pivot, then arrPartition() compares the other elements with the pivot and moves the smaller values to the left side. After that, the pivot is placed in its correct position, and quickSort() repeats the same process on the left and right parts of the array. The exact number of passes and comparisons can change because the numbers are randomly generated. The best-case time complexity is O(n log n) when the pivot divides the array into almost equal parts, while the worst case is O(n^2) when the pivot creates very uneven parts. Once the process is finished, the array is fully sorted, and display() goes through all 100 elements again to show the final sorted list.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

INPUT

sortingAlgo.h

GRESOLA_HOA8PQ_SUPPLE > C sortingAlgo.h > ...

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

// display array:
template <typename T>
void display(T arr[], int size){
    for(int i = 0; i < size; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

// problem 1:
template <typename T> // function that will split the array
int arrPartition(T arr[], int low, int high){
    T pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++){
        if(arr[j] < pivot){
            i++;

            T temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    T temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}
```

```
35  template <typename T>
36  void insertionSort(T arr[], int low, int high){
37      for(int i = low + 1; i <= high; i++){
38          T temp = arr[i];
39          int j = i - 1;
40
41          while(j >= low && arr[j] > temp){
42              arr[j + 1] = arr[j];
43              j--;
44          }
45          arr[j + 1] = temp;
46      }
47  }
48
49  #endif
```

main.cpp

GRESOLA_HOA8PQ_SUPPLE > main.cpp > ...

```
1   #include <iostream>
2   #include "sortingAlgo.h"
3
4   int main(){
5       // problem 1:
6       std::cout << "PROBLEM 1" << std::endl;
7
8       int numSet[] = {12, 25, 11, 42, 56, 75, 10};
9       int setSize = 7;
10
11      std::cout << "--- ORIGINAL UNSORTED ARRAY ---" << std::endl;
12      display(numSet, setSize);
13      std::cout << std::endl;
14
15      int pivot = arrPartition(numSet, 0, setSize - 1);
16      std::cout << "--- AFTER PARTITION IN QUICK SORT ---" << std::endl;
17      display(numSet, setSize);
18
19      std::cout << "PIVOT: " << numSet[pivot] << std::endl;
20
21      // this part will sort the left sublist using insertion sort:
22      insertionSort(numSet, 0, pivot - 1);
23      // this part will sort the right sublist using insertion sort:
24      insertionSort(numSet, pivot + 1, setSize - 1);
25
26      std::cout << "--- AFTER SORTING LEFT AND RIGHT SUBLISTS ---" << std::endl;
27      display(numSet, setSize);
28
29      return 0;
30  }
```

OUTPUT

```
PROBLEM 1
--- ORIGINAL UNSORTED ARRAY ---
12 25 11 42 56 75 10

--- AFTER PARTITION IN QUICK SORT ---
10 25 11 42 56 75 12
PIVOT: 10
--- AFTER SORTING LEFT AND RIGHT SUBLISTS ---
10 11 12 25 42 56 75
```

EXPLANATION

Yes, we can use other sorting algorithms to sort the left and right sublists after partitioning. In my code, I demonstrated this by using Insertion Sort after using the partition method from Quick Sort. I used the array 12, 25, 11, 42, 56, 75, 10. The arrPartition() function uses 10 as the pivot and separates the values into a left sublist and a right sublist. After the partition, I used insertionSort() to sort the left sublist and right sublist separately.

The final sorted array is:
10, 11, 12, 25, 42, 56, 75

This shows that we do not have to use Quick Sort again after partitioning. We can use another sorting algorithm, such as Insertion Sort, to sort the two sublists.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

INPUT

sortingAlgo.h

```
49  //problem 2:
50  template <typename T>
51  void merge(T arr[], int left, int middle, int right){
52      T temp[100];
53      int i = left;
54      int j = middle + 1;
55      int k = 0;
56
57      while( i <= middle && j <= right){
58          if(arr[i] <= arr[j]){
59              temp[k] = arr[i];
60              i++;
61          }else{
62              temp[k] = arr[j];
63              j++;
64          }
65          k++;
66      }
67      while(i <= middle){
68          temp[k] = arr[i];
69          i++;
70          k++;
71      }
72      while(j <= right){
73          temp[k] = arr[j];
74          j++;
75          k++;
76      }
77      for(i  = left, k = 0; i <= right; i++, k++){
78          arr[i]= temp[k];
79      }
80  }
```

```
81
82  template <typename T>
83  void mergeSort(T arr[], int left, int right){
84      if(left < right){
85          int middle = (left + right) / 2;
86          mergeSort(arr, left, middle);
87          mergeSort(arr, middle + 1, right);
88          merge(arr, left, middle, right);
89
90      }
91  }
92
93  // problem 2: quick sort
94  template <typename T>
95  void quickSort(T arr[], int low, int high){
96      if (low < high){
97          int pivot = arrPartition(arr, low, high);
98          quickSort(arr, low, pivot - 1);
99          quickSort(arr, pivot + 1, high);
100     }
101 }
102
103
104 #endif
```

main.cpp

```
30      // problem 2:
31      std::cout << "PROBLEM 2" << std::endl;
32      const int SIZE = 15;
33      int num[SIZE] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
34
35      // seperate copies for different sorting algo:
36      int mergeNum[SIZE];
37      int quickNum[SIZE];
38
39      for(int i = 0; i < SIZE; i++){
40          mergeNum[i] = num[i];
41          quickNum[i] = num[i];
42      }
```

```
44      // implementing merge sort:
45      std::cout << "--- MERGE SORT ---" << std::endl;
46      std::cout << "Before: ";
47      display(mergeNum, SIZE);
48
49      mergeSort(mergeNum, 0, SIZE - 1);
50      std::cout << "After: ";
51      display(mergeNum, SIZE);
52      std::cout << std::endl;
53
54      // implementing quick sort:
55      std::cout << "--- QUICK SORT ---" << std::endl;
56      std::cout << "Before: ";
57      display(quickNum, SIZE);
58
59      quickSort(quickNum, 0, SIZE - 1);
60      std::cout << "After: ";
61      display(quickNum, SIZE);
62
63      return 0;
64  }
```

OUTPUT

```
PROBLEM 2
--- MERGE SORT ---
Before: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
After: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93

--- QUICK SORT ---
Before: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74
After: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
```

EXPLANATION

For the given array, I would choose Quick Sort because the values are unsorted and Quick Sort can divide them into smaller parts using a pivot. This makes the sorting process more organized and efficient. In my code, I tested both Merge Sort and Quick Sort using the same set of values. Both were able to sort the array correctly into:

4, 12, 19, 25, 29, 30, 34, 43, 44, 48, 53, 67, 74, 87, 93

The reason why Merge Sort and Quick Sort can have O(n log n) is because both algorithms reduce the problem into smaller parts. Merge Sort keeps dividing the array into halves, while Quick Sort divides it based on the pivot. If the divisions are balanced, there will be around log n levels, and each level works with about n values, giving O(n log n).

I chose Quick Sort because it can be very efficient when the pivot divides the values into fairly balanced parts. However, its performance still depends on the pivot. If the pivot creates very uneven parts, Quick Sort can reach a worst-case time complexity of O(n^2).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly and ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction, code assistance, and explanation of concepts.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used Grammarly for grammar improvement and ChatGPT for code assistance and concept explanations. These tools did not replace my original thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.